CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler `0x` önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Yusuf Mert Turan
Öğrenci numarası (SID)	5001240003 (programda son 9 hane: 1240003)
E-posta	yusufmertturan@gmail.com
Seçilen görev (G1–G5)	G4 Sıralama ve medyan
Seçilen ISA ve sürümü (ör. RV32IM)	RISC-V RV32IM (-march=rv32im -mabi=ilp32). Gerekçe: yönergenin önerdiği ISA; sabit 32 bit instruction uzunluğu ve düzenli formatlar (R, I, S, B, U, J) elle kodlamayı kolaylaştırır; M uzantısı çarpmayı (Y4) tek instruction olarak sağlar; RARS ve riscv64-elf-gcc ile araç zinciri hazırdır.
Derleyici ve sürümü	riscv64-elf-gcc 15.2.0 (GCC), -O0 ve -O1
Simülatör / araçlar	RARS 1.6, riscv64-elf-objdump, tools/gcc2rars.py (GCC çıktısını RARS'a çeviren betik)
Git deposu adresi	https://github.com/Jopseps/computer-organisation-project

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	5001240003 → son hane 3	3
D2	SID'in son iki hanesi	son iki hane 03	03 (= 3)
S	SID'in hanelerinin toplamı	5+0+0+1+2+4+0+0+0+3 (tam	15

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
		numaranın 10 hanesi)	
N	24 + (D2 mod 25)	24 + (3 mod 25) = 24 + 3	27
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	d = 3 satırı	256 B / 32 B line / 2-way set-assoc. / 8 line / 4 set
Write policy	d çift → write-back, d tek → write-through	d = 3 tek	write-through
k	(S mod 7) + 3	(15 mod 7) + 3 = 1 + 3	4
Interrupt türü	S çift → Timer, S tek → I/O	S = 15 tek	I/O
f	1.0 + 0.1 × d GHz (varsayım)	1.0 + 0.1 × 3	1.3 GHz (varsayım)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	T1 = 1; T2 = 20 + 5 × (3 mod 4) = 20 + 15	T1 = 1 cycle; T2 = 35 cycle (varsayım)
Nc	2 + (S mod 7) (varsayım)	2 + (15 mod 7) = 2 + 1	3 (varsayım)
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	G4: SID (1240003) tek → azalan	Sıralama yönü: azalan

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	5	$\log_2$(32 B line) = 5
Line (r) veya set (d)	2	set (d): $\log_2$(4 set) = 2
Tag (s − r veya s − d)	25	tag = 32 − 2 − 5
Toplam	32	25 + 2 + 5 = 32 (RV32, 32-bit adres)

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☐ G1 Biyomedikal sinyal alarmı
- ☐ G2 Sınav notu dağılımı
- ☐ G3 Mesaj şifreleme ve bütünlük
- ☒ G4 Sıralama ve medyan
- ☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden):

> Program, N = 27 elemanlı global arr[] dizisini SID'den başlayan LCG ile doldurur (R = 1000, B = 0). Dizi, iç içe iki döngüyle yerinde bubble sort ile sıralanır; SID tek olduğu için yön azalandır. Karşılaştırma should_swap(a, b) fonksiyonunda yapılır; böylece kernel'de fonksiyon çağrısı ve dönüşü görünür, yer değiştirme sayısı global swaps değişkeninde sayılır. Bubble sort seçildi, çünkü basittir, yer değiştirme sayısı kolay izlenir ve arr[] üzerinde çok sayıda load/store üretir. Sıralamadan sonra medyan arr[(N−1)/2] elemanından (0x3FF maskesiyle), en büyük ve en küçük değer uçlardaki elemanlardan okunur ve global değişkenlerde tutulur.

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloka yapıştırılır (25–60 satır):

```
 1  /* Yusuf Mert Turan, 5001240003, G4 */
 2  #define N 27
 3
 4  const int SID = 1240003;
 5  int arr[N];
 6  int median, minV, maxV, swaps;
 7
 8  int should_swap(int a, int b){
 9      if(a < b){
10          return 1;
11      }
12      return 0;
13  }
14
15  int main(){
16      unsigned int x = SID;
17      for(int i = 0; i < N; i++){
18          x = 1664525 * x + 1013904223;
19          arr[i] = (x >> 16) % 1000;
20      }
21
22      for(int i = 0; i < N - 1; i++){
23          for(int j = 0; j < N - 1 - i; j++){
24              if(should_swap(arr[j], arr[j + 1])){
25                  int t = arr[j];
26                  arr[j] = arr[j + 1];
27                  arr[j + 1] = t;
28                  swaps++;
29              }
30          }
31      }
32
33      median = arr[(N - 1) / 2] & 0x3FF;
34      maxV = arr[0];
35      minV = arr[N - 1];
36
37      return 0;
38  }
```

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	5, 17–20, 22–31	arr[N] global (N = 27); doldurma döngüsü (17–20) ve sıralama döngüleri (22–31) diziyi iki kez geçer.
Y2	Koşullu dallanma	9, 24	should_swap içindeki if

#	Zorunlu yapı	Satır no	Açıklama
			(9) ve sıralamadaki if (24) koşullu dallanmadır.
Y3	Parametreli, değer döndüren fonksiyon çağrısı	8–13, 24	should_swap(a, b) iki parametre alır, 0 veya 1 döndürür; satır 24'te çağrılır.
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	18, 19, 33	Çarpma ve toplama (18), shift ve mod (19), bit işlemi AND (33).
Y5	Dizi elemanına store	19, 26, 27	Doldururken arr[i] (19); yer değiştirirken arr[j] ve arr[j+1] yazılır (26–27).
Y6	const int SID ve hesapta kullanımı	4, 16, 18	const int SID = 1240003 (4); x = SID üretecin başlangıç değeri (16), dizinin tüm değerlerini etkiler (18).
Y7	Sonucun tutulduğu global değişken(ler)	6	median, minV, maxV, swaps global değişkenlerdir (6); sonuçlar 28 ve 33–35. satırlarda yazılır.

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
median	0x10010004	494	build/rars_O0_result.txt (RARS komut satırı çıktısı)
minV	0x10010008	4	build/rars_O0_result.txt (RARS komut satırı çıktısı)
maxV	0x1001000C	953	build/rars_O0_result.txt (RARS komut satırı çıktısı)
swaps	0x10010010	173	build/rars_O0_result.txt (RARS komut satırı çıktısı)

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı	24 (çağrı ve if), 8–13 (should_swap)
Başlangıç ve bitiş adresi (hex)	0x40012C – 0x400134 (arada should_swap: 0x400000 – 0x400040)
Dinamik instruction sayısı	18
Hangi döngü iterasyonu (ör. i = 3)	i = 0, j = 0 (arr[0] = 575, arr[1] = 383; yer değiştirme yok). Liste: kernel_M1.md

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

YZ kullanım beyanı

YZ kullanım beyanı. Kullanılmadıysa ilk satıra "Kullanılmadı" yazın. Beyan edilmemiş kullanım akademik dürüstlük ihlalidir.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
Claude Code (Claude Sonnet 5.5)	Adım 0	Parametrelerin hesaplanması ve açıklanması; S'nin tam numaradan mı 9 haneden mi hesaplanacağı belirsizliğinin tespiti	So why do we only use 9 digit numbers? Is it because of the 10 bit thing?	Formüller yönergeyle karşılaştırıldı, Python ile yeniden hesaplandı
Claude Code; RARS 1.6; riscv64-elf-gcc 15.2.0	Adım 1 (1.5, 1.6)	Araç kurulumu, GCC çıktısını RARS'a çeviren betik (tools/gcc2rars.py), kernel seçimi	Aight explain the rars stuff and give me docs and stackoverflows about it.	Adresler objdump ile, bellek değerleri RARS ile karşılaştırıldı
Claude Code (Claude Sonnet 5.5)	Adım 1 (1.2, 1.3, 1.4)	C kodunun kontrolü ve tutanak tablolarının doldurulması	Aight check this code and complete if it has missing stuff. Tell me what you changed.	Kodu okudum ve anladım, değişkenleri ve yorum satırlarını kendim düzenledim (minV, maxV); derleyip RARS'ta çalıştırdım, sonuçları (494/4/953/173) kontrol ettim